Greedy and ILP: choosing backup pipes

A detailed explanation of slide 7 of the CLL798D final presentation, with a worked example, the maths, the code, and our results

Uday Pratap Singh · Hardik Kotwar · November 2026

The whole story in one minute

- Our network is a **tree**, so every pipe is a single point of failure. If one breaks, everything downstream loses heat.
- Adding one extra **backup pipe** makes a loop. Every tree pipe on that loop is now protected, because heat can go round the other way.
- We want to choose a few backup pipes that protect the most **expected unserved demand (EUD)** for the money.
- **Greedy** (used at the prelim): add pipes one at a time, each time picking the pipe with the best *EUD removed per euro*. Fast and simple, but no guarantee of the best answer.
- **ILP** (integer linear program, added after the prelim): write the choice as yes/no variables with a linear objective and linear constraints, and let a solver find the **guaranteed best** answer.
- **Result:** for 1 to 8 backup pipes, greedy is never more than **1.5 percentage points** below the ILP optimum. In Bremen it matches exactly for 1 to 6 pipes. So greedy is good enough.
- **Twist:** when we re-check the one case where the ILP beats greedy with the full physics model, the ILP's pair turns out more expensive and leaves the worst failure unprotected. Greedy's value-for-money choice was the better engineering decision.

Contents

1. The problem: why a tree needs backup pipes
2. Turning it into a coverage problem
3. The greedy method
4. The ILP formulation
5. How the solver finds the optimum: LP relaxation and branch and bound
6. Brute-force check with bitmasks
7. Results in our project
8. The twist: re-checking with the full physics model
9. Putting it together with the edge-swap tree
10. How to present it, and likely questions

1. The problem: why a tree needs backup pipes

In a tree there is exactly one path from the plant to every substation. So every pipe is a **bridge**: remove it and the network splits in two. If a pipe near the plant breaks, every substation beyond it is cut off until it is repaired (we assume 12 hours).

Real-world picture: a town with one road out. If that road is blocked, everyone is stuck. Build one ring road, and every road segment on that ring now has a detour.

We measure how bad each possible failure is with **expected unserved demand (EUD)**, in MWh per year:

```
EUD of pipe e  =  failure rate of e  ×  repair time  ×  demand cut off when e fails
network EUD    =  sum over all tree pipes
```

Failure rates come from published district-heating statistics (Valinčius et al., 2015): they grow with pipe length, are higher for small pipes (DN ≤ 150), and are adjusted for corrosion. A pipe near the plant carries a lot of demand, so it has a large EUD even if it rarely fails. In Stuttgart the base network's EUD is 430 MWh/yr, in Bremen 1,224 MWh/yr.

What a backup pipe does. Add a pipe between substations u and v that are not directly connected. It closes exactly one cycle: the new pipe plus the tree path from u to v. Every tree pipe on that path is now protected: if it fails, heat can reach the far side the other way round. Pipes not on the path stay unprotected.

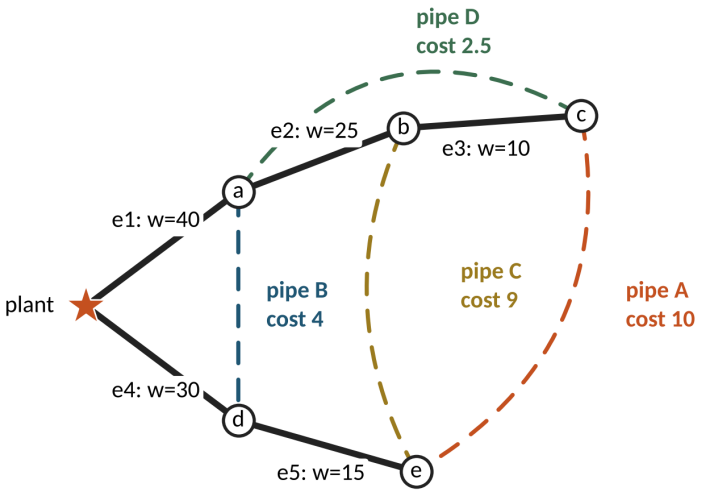
> **Key idea**
>
> A backup pipe (u, v) protects **exactly the tree pipes on the path from u to v**. Nothing more, nothing less. This one fact turns the problem into a clean counting problem.

2. Turning it into a coverage problem

Give each tree pipe e a **weight** w_e = its EUD. Each candidate backup pipe c "covers" the set of tree pipes on its cycle, and costs $cost_c$. A tree pipe is protected if **at least one** chosen backup pipe covers it. We want to choose backup pipes so that the total weight of protected tree pipes is as large as possible, using at most k pipes and a limited budget.

This is a textbook problem called **weighted maximum coverage**. In general it is NP-hard: no fast algorithm is known that always finds the best answer for large inputs. That is why the prelim used a heuristic (greedy) and why we needed to check it.

2.1 A toy example we will use throughout



Toy network: a plant, five substations, five tree pipes e1 to e5 with their EUD weights w, and four candidate backup pipes A to D (dashed) with their costs.

Backup pipe	Closes the cycle through	Tree pipes it covers	EUD covered	Cost
A (c to e)	c–b–a–plant–d–e	e1, e2, e3, e4, e5	120 (all)	10
B (a to d)	a–plant–d	e1, e4	70	4
C (b to e)	b–a–plant–d–e	e1, e2, e4, e5	110	9
D (c to a)	c–b–a	e2, e3	35	2.5

Total EUD is 40 + 25 + 10 + 30 + 15 = 120. The question: **with at most 2 pipes and a budget of 10, which pipes protect the most EUD?**

2.2 In our real project

	Toy	Our project (40 substations)
Tree pipes (things to protect)	5	40
Candidate backup pipes	4	780 (every pair of substations not already joined)
Weight w_e	made up	EUD of each tree pipe from the reliability model
Cost of a backup pipe	made up	annual cost of a pipe sized to carry the worst stranded load (DN600), times its terrain-weighted length, as % of the network's annual cost
Pipes allowed	2	1 to 8

3. The greedy method

3.1 The rule

Greedy builds the answer one pipe at a time and never goes back. At each step it looks at every candidate pipe that still fits in the budget and computes its **score**:

```
score(c) = (extra EUD protected if we add c now) / cost(c)
```

It adds the pipe with the highest score, then repeats. "Extra" matters: a pipe's value depends on what is already chosen. This is exactly the rule in our prelim code (*greedy_augment* in *src/reliability.py*), which re-runs the full reliability model for every candidate at every step.

C++ view

It is the same shape as any greedy loop you have written: pick the locally best option by a key, commit, repeat. No backtracking.

```
std::vector<int> chosen;  double spent = 0;
for (int step = 0; step < K; ++step) {
    int best = -1;  double bestScore = 0;
    for (int c = 0; c < numCandidates; ++c) {
        if (isChosen(c) || spent + cost[c] > budget) continue;
        double gain  = covered(chosen + c) - covered(chosen);   // EXTRA EUD protected
        double score = gain / cost[c];                          // value for money
        if (score > bestScore) { bestScore = score; best = c; }
    }
    if (best < 0) break;
    chosen.push_back(best);  spent += cost[best];
}
```

3.2 Greedy on the toy example (budget 10, at most 2 pipes)

Step 1. Nothing is protected yet, so each pipe's extra gain is its full coverage:

Pipe	Extra EUD	Cost	Score = gain / cost
A	120	10	12.0
B	70	4	**17.5 (best)**
C	110	9	12.2
D	35	2.5	14.0

Greedy picks **B**. Spent 4, budget left 6.

Step 2. Only pipes costing ≤ 6 still fit: A (10) and C (9) are now too expensive. D still fits. B already protects e1 and e4, so D's extra gain is e2 + e3 = 35, score 14. Greedy picks **D**.

Greedy answer: B + D, protecting 105 of 120, cost 6.5.

3.3 Where greedy goes wrong

The best answer within budget 10 is **pipe A alone**: it protects all 120 for cost 10. Greedy never takes it because in step 1 its score (12) is lower than B's (17.5), and after B is chosen A no longer fits the budget. Greedy is **short-sighted**: it grabs the best value now and cannot undo it.

Notice that greedy is not wrong about value for money. B + D gives 105 for 6.5 (16 per unit) while A gives 120 for 10 (12 per unit). If the budget had been 6.5, greedy's answer would be the optimum. That is why, in our project, we compare greedy and the ILP **at the same budget greedy used**: it asks "could the same money have bought more protection?"

3.4 Why greedy usually works well: diminishing returns

Coverage has **diminishing returns** (the maths name is **submodular**): a pipe is worth less once its tree pipes are already covered. In the toy example, B is worth 70 on its own, still 70 after D (they cover different pipes), but **0** after A (A already covers e1 and e4).

For problems with this property there is a classic theorem: for the plain version (pick any k pipes, no budget), greedy that picks the largest gain each time always reaches at least **1 − 1/e ≈ 63%** of the optimum, even in the worst possible case. In practice it is usually far closer. With a budget and the "gain per cost" rule there is no such guarantee on its own; a known fix is to also compare with the best single pipe (Khuller, Moss and Naor, 1999). The 63% is a worst-case floor, which is exactly why we checked our real case with an exact method.

4. The ILP formulation

4.1 What an ILP is

An **integer linear program** has three parts: **decision variables** that must be whole numbers (here 0 or 1, meaning no or yes), an **objective** to maximise or minimise, and **constraints**. Both the objective and the constraints must be **linear**: variables are only multiplied by constants and added. No products of variables, no squares, no if-statements. A solver program then finds the guaranteed best solution, and can prove it is the best.

4.2 Variables

- x_c = 1 if we build backup pipe c, 0 if not. One per candidate: 780 in our project, 4 in the toy.
- y_e = 1 if tree pipe e is protected, 0 if not. One per tree pipe: 40 in our project, 5 in the toy.

4.3 Objective and constraints

```
maximise    Σ_e  w_e · y_e                          total EUD protected

subject to  y_e ≤ Σ (x_c over pipes c that cover e)    for every tree pipe e
            Σ_c  x_c ≤ k                              at most k backup pipes
            Σ_c  cost_c · x_c ≤ budget                spend at most the budget
            x_c, y_e ∈ {0, 1}
```

4.4 The one clever line: $y_e \le \Sigma x_c$

We want "e is protected **if** at least one chosen pipe covers it". That is a logical OR, and an ILP cannot write OR directly. The inequality does the job:

- If **no** chosen pipe covers e, the right side is 0, so $y_e \le 0$ forces $y_e = 0$: not protected.
- If **one or more** chosen pipes cover e, the right side is ≥ 1, so y_e is allowed to be 1. Since the objective wants y_e as large as possible, the solver sets it to 1.
- Covering a pipe twice does not count twice, because y_e can never exceed 1. That is exactly the diminishing-returns behaviour from Section 3.4.

> **C++ view**
>
> In C++ you would write `protected[e] = x[A] || x[C];`. The ILP cannot use `||`, so it writes `y[e] <= x[A] + x[C]` and lets the maximisation push y[e] up to 1 whenever allowed. This trick of replacing logic with inequalities is called **linearisation**, and it is the standard way to put if/or rules into an ILP.

4.5 The toy ILP written out in full (budget 10, at most 2 pipes)

```
maximise   40·y1 + 25·y2 + 10·y3 + 30·y4 + 15·y5

subject to y1 ≤ xA + xB + xC          (e1 covered by A, B, C)
           y2 ≤ xA + xC + xD          (e2 covered by A, C, D)
           y3 ≤ xA + xD               (e3 covered by A, D)
           y4 ≤ xA + xB + xC          (e4 covered by A, B, C)
           y5 ≤ xA + xC               (e5 covered by A, C)
           xA + xB + xC + xD ≤ 2
           10·xA + 4·xB + 9·xC + 2.5·xD ≤ 10
           all variables 0 or 1
```

The solver returns xA = 1, all y = 1: **protect 120 with pipe A**, the true optimum that greedy missed.

4.6 Our real ILP

- **820 binary variables**: 780 x's and 40 y's.
- **42 constraints**: 40 coverage lines (one per tree pipe), 1 for the number of pipes, 1 for the budget.
- The budget is set to **what greedy spent** for that number of pipes, so we compare like with like.
- Solved with **HiGHS**, an open-source optimisation solver, called from Python through *scipy.optimize.milp*, with the optimality gap set to zero so it must prove the answer is the best.
- Time: under 1 second for every k from 1 to 8 in both cities.
- Code: *run_final_G.py*, function *ilp()*.

5. How the solver finds the optimum: LP relaxation and branch and bound

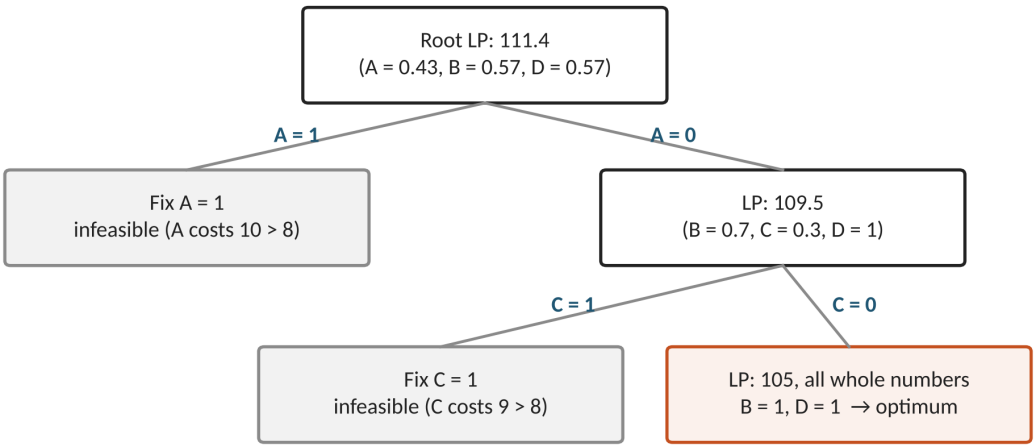
Trying every combination is hopeless for large k: with 780 pipes, choosing 8 gives more than 10^{18} combinations. The solver uses two ideas instead.

5.1 LP relaxation: an easy problem that gives an upper bound

Temporarily allow the variables to take **any value between 0 and 1**, like 0.43, instead of only 0 or 1. This "relaxed" problem is a plain linear program, which can be solved very fast. Its answer can never be worse than the true answer, because it has more freedom. So it gives an **upper bound**: no real (0/1) solution can beat it.

5.2 Branch and bound: split, solve, prune

If the relaxed answer has a fractional variable, pick it and **branch**: solve one version with that variable fixed to 1 and one with it fixed to 0. Keep the best whole-number solution found so far. Any branch whose upper bound is already below that best solution, or which is impossible, is **pruned**: thrown away without exploring it. Here is the toy example with budget 8:



Branch and bound on the toy example (budget 8, at most 2 pipes). Grey boxes are pruned because they are impossible. The orange box is a whole-number solution, and nothing else is left to explore, so it is proven optimal.

- **Root:** the relaxed problem says 111.4, using fractions of pipes A, B and D. So the true optimum is at most 111.4.
- **Branch on A.** A = 1 is impossible (A costs 10, budget 8). A = 0 gives a relaxed bound of 109.5, still with a fraction (C = 0.3).
- **Branch on C.** C = 1 is impossible (C costs 9). C = 0 gives 105 with all variables whole numbers: build B and D.
- Nothing is left to explore, so **105 is proven optimal**. Here greedy also finds B + D, so greedy would be optimal for this budget.

C++ view

Branch and bound is a backtracking DFS with pruning, the same idea as solving 0/1 knapsack by recursion with an upper-bound cut-off. The LP relaxation plays the role of the "optimistic estimate" you compare with your best answer so far before going deeper.

```
void solve(Node n) {
    double bound = solveLPRelaxation(n);          // optimistic estimate
    if (infeasible(n) || bound <= bestSoFar) return; // prune this whole subtree
    if (allWholeNumbers(n)) { bestSoFar = bound; bestSolution = n; return; }
    int v = pickFractionalVariable(n);
    solve(n.with(v, 1));                          // branch: build it
    solve(n.with(v, 0));                          // branch: don't build it
}
```

6. Brute-force check with bitmasks

Before writing the ILP we checked greedy for up to 3 pipes by trying **every** combination. That is 780 singles, 303,810 pairs and 78,788,060 triples per city. This was only practical because of a trick: each candidate pipe is stored as a **40-bit mask** saying which tree pipes it covers.

C++ view

Combining pipes is a bitwise OR, and the EUD protected is a weighted popcount, so each combination costs a few machine instructions. (We used numpy arrays of uint64 and a 10-bit lookup table instead of the inner loop, for speed.)

```
uint64_t mask[780];       // bit e is set if pipe covers tree edge e
double gain(uint64_t m) { // EUD protected by a set of pipes
    double s = 0;
    for (int e = 0; e < 40; ++e) if (m >> e & 1) s += w[e];
    return s;
}
// for all i < j < k:  best = max(best, gain(mask[i] | mask[j] | mask[k]));
```

For k = 1, 2 and 3 the ILP gives **exactly** the same optimum as the brute force in both cities, which confirms the ILP is set up correctly. For k = 4 to 8 brute force is too slow, and only the ILP can answer.

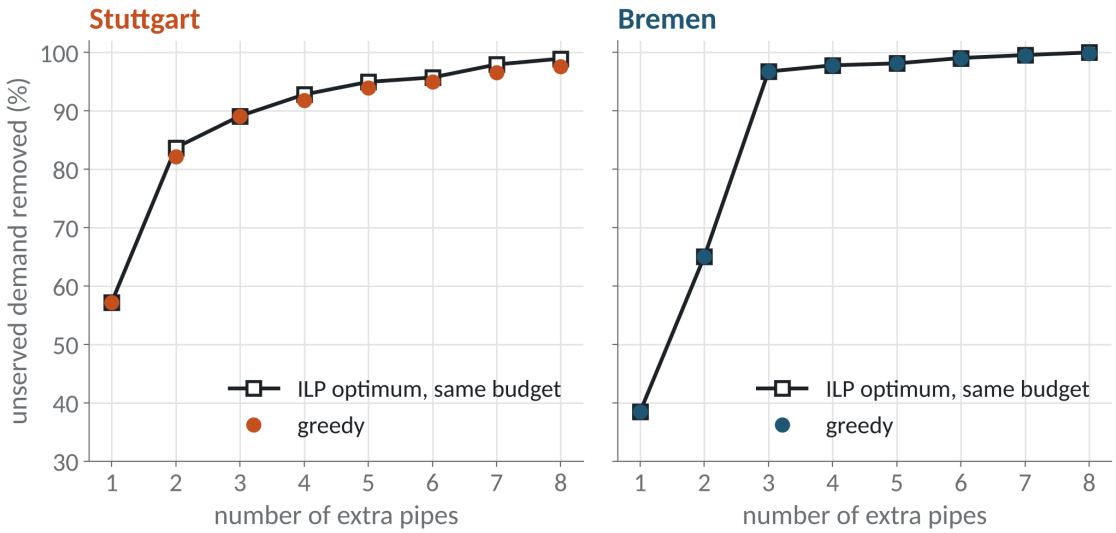
7. Results in our project

Both cities, 40 substations, prelim clustering. "Cost" is the estimated extra annual cost of the backup pipes, as a percentage of the network's annual cost. "ILP, same budget" is the best possible with the same number of pipes and the same money greedy spent.

Pipes	Stuttgart greedy	Stuttgart ILP	Gap	Bremen greedy	Bremen ILP	Gap
1	57.2% (cost +4.2%)	57.2%	0.00	38.5% (cost +3.4%)	38.5%	0.00
2	82.2% (cost +12.6%)	83.7%	1.51	65.0% (cost +7.0%)	65.0%	0.00
3	89.1% (cost +16.8%)	89.1%	0.00	96.7% (cost +12.8%)	96.7%	0.00
4	91.8% (cost +19.8%)	92.8%	1.02	97.8% (cost +16.3%)	97.8%	0.00
5	93.9% (cost +24.5%)	95.0%	1.02	98.1% (cost +18.3%)	98.1%	0.00

Pipes	Stuttgart greedy	Stuttgart ILP	Gap	Bremen greedy	Bremen ILP	Gap
6	95.0% (cost +28.1%)	95.7%	0.76	99.0% (cost +24.9%)	99.0%	0.00
7	96.6% (cost +38.5%)	97.9%	1.34	99.5% (cost +28.6%)	99.6%	0.07
8	97.6% (cost +45.1%)	98.9%	1.33	99.8% (cost +34.2%)	100.0%	0.15

Gaps are in percentage points of EUD protected, computed before rounding. A gap of 1.5 means greedy protected 82.2% where the best possible was 83.7%.



Black line: ILP optimum for the same budget. Dots: greedy. The dots sit on or just under the line.

What to read from it:

- **Greedy is never more than 1.5 points below the optimum**, for 1 to 8 pipes, in both cities.
- **Bremen:** greedy is exactly optimal for 1 to 6 pipes, and within 0.15 points at 7 and 8.
- **Stuttgart:** the largest gap is 1.5 points at 2 pipes; the gap is between 0.8 and 1.3 points from 4 pipes on.
- **The first pipe does most of the work:** 57% of EUD in Stuttgart and 39% in Bremen with one pipe. After 3 pipes, extra pipes add little: the diminishing returns from Section 3.4.

Why "same budget" and not "unlimited budget"? Without a budget, the ILP happily picks very long, expensive pipes. In Stuttgart one unconstrained pipe protects 78% of EUD, but costs about 21% extra instead of greedy's 4%. That is not a fair comparison: it answers a different question. The useful question is whether greedy wasted money, and the answer is: hardly.

8. The twist: re-checking with the full physics model

The coverage model behind the ILP is a simplification:

- Each tree pipe's weight w_e is computed once, on the original tree.
- The cost of a backup pipe is an estimate (a pipe sized for the worst stranded load, times its length). In reality, adding a loop also changes the sizes of other pipes, because each side of a loop must carry the full load when the other side fails (N-1 sizing).

So for the one case where the ILP clearly beat greedy (Stuttgart, 2 pipes), we built both networks and costed them with the **full model**, including N-1 pipe sizing:

Stuttgart, 2 backup pipes	Extra cost (full model)	EUD removed	Demand served after worst failure
Greedy pair	+13.2%	82.2%	68%
ILP pair (same estimated budget)	+30.5%	83.7%	0%

The ILP's pair protects slightly more *expected* demand, but it costs more than twice as much once sized properly, and it leaves the **single worst failure unprotected**: if the main pipe next to the plant fails, nobody gets heat. Greedy's pair protects that trunk.

The same happens with the unconstrained single best pipe in Stuttgart: it removes 78% of EUD but leaves 0% served after the worst failure and costs +55% with full sizing.

Key idea

An exact optimiser is only as good as the model you give it. EUD (an average over all failures) and N-1 (the worst single failure) are different goals. Greedy's preference for cheap, high-value pipes happened to protect the trunk near the plant, which is what matters most in practice. So the ILP did its job: it showed greedy is near-optimal on the coverage model, and the full re-check showed greedy's choice is the better engineering decision.

9. Putting it together with the edge-swap tree

Slide 7 also combines the two new ideas: first improve the tree with edge swaps (slide 4), then add one greedy backup pipe. Stuttgart, mean of 10 clusterings, seasonal costing, full N-1 sizing:

Design	Cost (EUR/MWh)	Demand served after worst failure
Terrain MST	17.25	30%
Terrain MST + 1 backup pipe	19.39	68%
Edge-swap tree	16.24	51%
Edge-swap tree + 1 backup pipe	**18.45**	**67%**

- Edge-swap tree + 1 pipe has the same reliability as MST + 1 pipe and is about **5% cheaper**. Bremen: 15.11 against 15.95 EUR/MWh.
- The edge-swap tree alone is already more robust than the MST (51% vs 30%): it is shallower, so less demand depends on any single pipe.

10. How to present it, and likely questions

How to say it on stage

"A tree has no backup: if one pipe breaks, everything after it loses heat. At the prelim we added backup pipes greedily, always taking the best value for money, and we promised to check this against an exact method. On a tree, a new pipe protects exactly the pipes on the cycle it closes, so this is a maximum coverage problem. We wrote it as an integer linear program with one yes/no variable per candidate pipe, and maximised the protected demand for the same budget as greedy. For 1 to 8 pipes, greedy is never more than 1.5 points below the optimum. And the edge-swap tree plus one backup pipe is as reliable as the MST plus one, about 5 percent cheaper."

Q. What is an ILP?

An optimisation where the variables must be whole numbers, here 0 or 1 for each backup pipe, with a linear objective and linear constraints. A solver finds the proven best answer.

Q. What are your variables and constraints?

780 binary x's (build pipe or not) and 40 binary y's (tree pipe protected or not). Maximise the sum of EUD weights of protected pipes. y_e ≤ sum of x over pipes covering e, at most k pipes, and cost at most greedy's budget.

Q. Why does y_e ≤ Σ x_c work?

If no chosen pipe covers e, y_e is forced to 0. If at least one does, y_e may be 1 and the maximisation sets it to 1. It turns the logical OR into a linear inequality.

Q. Why not just use the ILP and forget greedy?

We could here, because the problem is small (820 variables, under a second). Greedy is useful because it is simple, needs no solver, and gives the whole cost-reliability curve step by step. The ILP is the check that greedy is good.

Q. Is greedy guaranteed to be good?

For picking k pipes by largest gain there is a classic guarantee of at least 63% (1 − 1/e) of the optimum, because coverage has diminishing returns. In our case it was within 1.5 points, much better than that worst case.

Q. Why compare at the same budget?

Because without a budget the ILP picks very long, expensive pipes. Same budget asks the fair question: could the same money have bought more protection? Barely: at most 1.5 points.

Q. How do you know the ILP is right?

For 1 to 3 pipes we also tried every combination by brute force (78.8 million triples per city) and got exactly the same optimum.

Q. Then why does the ILP pair cost more?

The ILP uses a simplified cost estimate. With full N-1 pipe sizing its Stuttgart 2-pipe choice costs +30.5% vs greedy's +13.2%, and it leaves the worst failure unprotected. The optimiser is only as good as the model.

Q. Why EUD and not just N-1?

EUD weighs every failure by how likely it is and how much heat it cuts off, so it uses all the information. N-1 only looks at the single worst case. We report both, and the twist in Section 8 shows why both matter.

Q. What is the solver doing inside?

Branch and bound: it solves a relaxed version with fractional variables to get an upper bound, splits on a fractional variable (build it / don't), and prunes branches that cannot beat the best solution found so far.